Lab 4 – Camera Calibration

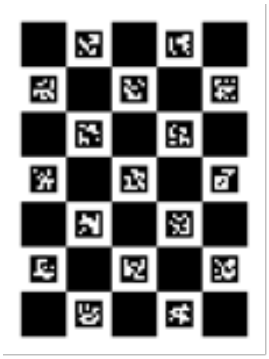
1 Introduction

In this lab camera calibration will be performed to determine the intrinsic parameters of the camera. The calibrated camera as well as ArUco fiducial markers will then be used to determine the intrinsic parameters of the camera and by inverse, the pose of the ArUco board. The virtual robot created in the previous lab will then be transformed to the origin of the ArUco board coordinate system to create an augmented reality interface. Example OpenCV code for working with ArUco markers and boards can be found on the OpenCV object detection documentation website:

https://docs.opencv.org/4.x/d2/d64/tutorial_table_of_content_objdetect.html

2 Experiment

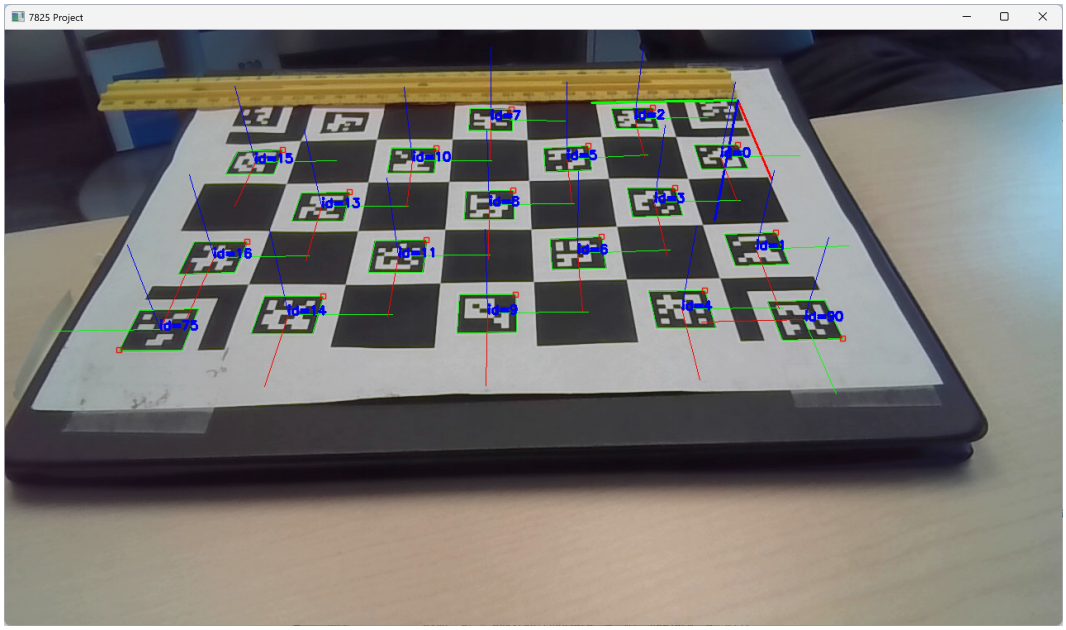
2.1 Calibration Part A



A) Calibrate your laptop camera or webcam using the MATLAB camera calibration toolbox. Determine the camera principal point and focal length in pixels.

B) Print the ChArUco checkboard pattern and calibrate the camera using the ChArUco calibration technique. Determine the camera principal point and focal length in pixels. You must tape to a flat surface to get an accurate result. You must also accurately measure the size of the squares and ArUco markers and enter these values into the calibration procedure.

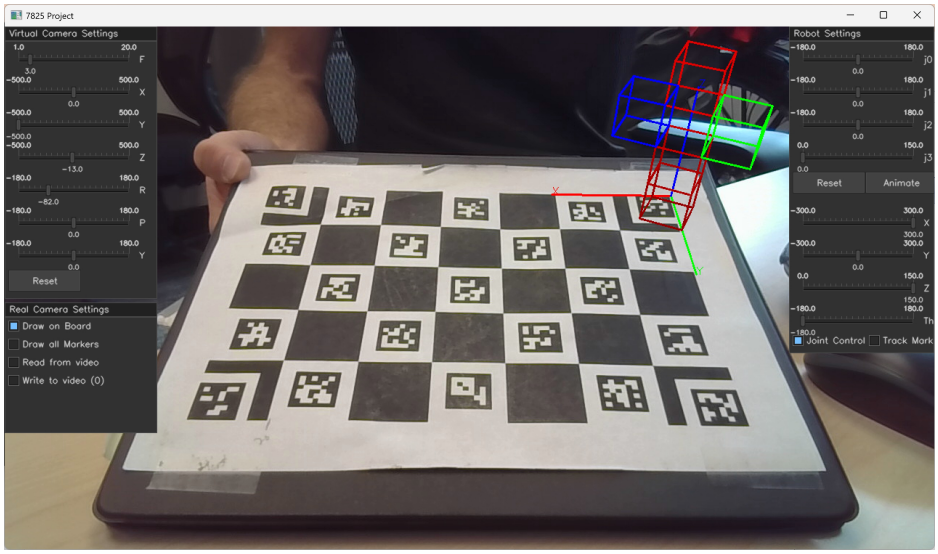
C) Capture the ChArUco board with the webcam and determine the pose of the board. Use the Track Board checkbox to turn on/off a link of the X/Y/Z values to the virtual camera slider bars for a real-time display of the position. Draw the coordinate frame of the board overlaid on the webcam image. Axis lengths should be equal to two squares, measure with a ruler to verify.



2.2 Calibration Part B

A) Using the calibrated camera properties (intrinsic and distortion matrixes) and the pose of the board, transform the virtual robot to position it at the origin of the ChArUco board. Use the `projectPoints` function in OpenCV to transform the 3D robot points to 2D image points. Note the input points must be in a `vector<Point3f>` format while the output is in `vector<Point2f>` format for this function. You may need to rotate the robot to match the board coordinate system as the Z axis points into the board.

B) Add a slow rotation about the Z axis of the robot to show motion over time.




```
// Convert from Rodrigues 3x1 to Rotation matrix 3x3
Rodrigues(rvec, R);
// Create Homogeneous transformation from R
T = createHT(tvec, R);
// Create HT to rotate 180 about X and 90 about Z
Mat rotation = createHT(Vec3d(0, 0, 0), Vec3d(CV_PI, 0, CV_PI / 2));
// Board Z is into the board so rotate 180 about X & 90 about Z
T = T * rot;
// Copy rotation matrix from HT back into rotation matrix R
R.at<double>(0, 0) = _board._cT.at<float>(0, 0);
R.at<double>(1, 0) = _board._cT.at<float>(1, 0);
R.at<double>(2, 0) = _board._cT.at<float>(2, 0);
R.at<double>(0, 1) = _board._cT.at<float>(0, 1);
R.at<double>(1, 1) = _board._cT.at<float>(1, 1);
R.at<double>(2, 1) = _board._cT.at<float>(2, 1);
R.at<double>(0, 2) = _board._cT.at<float>(0, 2);
R.at<double>(1, 2) = _board._cT.at<float>(1, 2);
R.at<double>(2, 2) = _board._cT.at<float>(2, 2);
// Convert from R to Rodrigues
Rodrigues(R, rvec);
```

NOTE 1: If the webcam loads slowly try the following:

```
cv::VideoCapture vid;
vid.open(cam_id, CAP_DSHOW);
```

NOTE 2: Run in RELEASE mode for real-time rendering